Portada

Una única semilla —el modelo de datos— de la que crece todo lo demás.

QUÉ DECIR

- Buenas tardes. Soy **Nicolás Rey Alonso** y presento mi TFG.
- «Framework backend–frontend para el desarrollo rápido de proyectos apoyado en un **modelo de datos en evolución**».
- Aplicado a los proyectos del **ITC** con el **Jardín Botánico Viera y Clavijo**. Se llama **UniWorks**.
- Tutores: **Rafael Juan Nebot Medina** (ITC) y **María Dolores Afonso Suárez** (Escuela).
- La planta es la idea: **una semilla, el modelo, y todo lo demás crece de ella**.

EN PANTALLA

La planta crece desde la semilla al entrar. Preséntate mientras crece; no esperes a que termine.

SI PREGUNTAN

¿Qué es UniWorks, en una frase?

Dos piezas unidas por un contrato HTTP: **uniback** (backend Python con FastAPI y PostgreSQL) y **ngt-gui** (librería de componentes Angular).

→ Empiezo por el problema que lo motiva todo.

El problema

Cuando el modelo cambia, todo lo demás se reescribe manualmente.

QUÉ DECIR

- El ITC hace con el Jardín Botánico apps de **datos botánicos**; su modelo **no es estable**: crece con la investigación y los estándares.
- Cada cambio va **en cascada**: backend (modelos, validación, endpoints) → comunicación (contratos, tipos) → frontend (formularios, tablas, vistas).
- Todo se reescribe **manualmente**, capa a capa, y queda **acoplado a un dominio**.
- Resultado: **caro de mantener** y **difícil de reutilizar** en el siguiente proyecto.

EN PANTALLA

Backend, Comunicación y Frontend caen desde «Modelo de datos» (≈ 1,3 s). Después el cursor escribe «a mano», lo borra y escribe «manualmente.» (acaba ≈ 4,5 s): cierra la idea cuando termine.

SI PREGUNTAN

¿No basta con OpenAPI/Swagger?

Describe **operaciones**, no presentación: el frontend seguiría escribiendo sus formularios a mano.

→ ¿Y si el modelo fuera lo único que se escribe?

La propuesta

El modelo de datos como única fuente de verdad.

QUÉ DECIR

- Solo se escribe el **modelo ORM** (SQLAlchemy).
- De él se **deriva** un **JSON Schema enriquecido** con pistas de presentación (`x-*`).
- Viaja por un **contrato uniforme** y, al otro lado, **se genera el formulario**, sin HTML por entidad.
- Dos piezas que encajan por el contrato: **uniback** (FastAPI + PostgreSQL) y **ngt-gui** (Angular).
- Un cambio en el modelo **se propaga solo**.

EN PANTALLA

El punto sale de «Modelo ORM» y enciende los nodos uno a uno hasta «Formulario» (≈ 3,3 s). Acompaña: «se deriva… se deriva… se genera».

SI PREGUNTAN

¿Y si quiero personalizar un campo?
Las pistas son **valores por defecto**: lo que el desarrollador declara a mano en la columna **siempre gana**.

→ Antes del diseño: qué recibí y qué aporto.

Punto de partida

Dos piezas útiles que no encajaban. Mi aportación: de la fase 1 a la fase 2.

QUÉ DECIR

- **Fase 0** (prácticas): app Angular monolítica de **NextGenDem**, con **178 métodos** de acceso al servidor escritos a mano.
- En las prácticas extraje sus componentes a una librería: **ngt-gui**.
- **Fase 1** (inicio del TFG): ngt-gui, aún **atada a su dominio**, y **uniback** del ITC: buen núcleo de modelos, primer envoltorio y fábrica CRUD para **≈ 20 entidades**; sin descubrimiento, sin pistas, sin plugins.
- **Mi aportación**: todo lo que separa la **fase 1** de la **fase 2**.

EN PANTALLA

Fase 0 → Prácticas → Fase 1 → Este TFG → Fase 2 entran de izquierda a derecha (≈ 1,9 s).

SI PREGUNTAN

¿Qué es tuyo y qué del ITC?

Del ITC: núcleo de modelos, primer envoltorio y primera fábrica CRUD (≈ 20 entidades). Mío: contrato universal y errores, descubrimiento, cliente genérico, pistas `x-*`, conversor a Formly, bundle versionado, plugins y micronúcleo.

→ Lo que me propuse conseguir.

Objetivos

Un framework reutilizable, extensible y desacoplado del dominio.

QUÉ DECIR

- Objetivo general: framework **reutilizable**, **extensible** y **desacoplado del dominio**.
- **1** API modular: envoltorio en toda la API, catálogo de errores, fábricas CRUD. **2** Modelo de datos: versionado e histórico.
- **3** Comunicación: descubrimiento y cliente genérico tipado. **4** Formularios automáticos con Formly. **5** Esquemas desde el ORM.
- Requisitos con **MoSCoW**: **10 funcionales** y **7 no funcionales** (reutilización, portabilidad, compatibilidad con lo heredado).

EN PANTALLA

Los números 1 → 5 aparecen en orden, cada uno con su objetivo (≈ 2 s). Nómbralos a su ritmo.

SI PREGUNTAN

¿Qué es MoSCoW?

Priorización en **Must / Should / Could**: separa lo imprescindible de lo deseable.

→ ¿No existe ya algo así?

Estado del arte

Los renderizadores de esquemas ya existen. Faltaba la cadena completa.

QUÉ DECIR

- **react-jsonschema-form** y **JSON Forms**: renderizan, pero en el cliente y con el esquema ya hecho.
- **OpenAPI / Swagger**: describe **operaciones**, no presentación.
- **Admins autogenerados** (tipo Django Admin): parten del ORM, pero **acoplan** interfaz y servidor.
- Mi aportación no es otro renderizador: es **completar la cadena** — ORM → esquema enriquecido → contrato versionado → capa genérica.
- Última fila: **UniWorks es el único que cubre las cuatro columnas**.

EN PANTALLA

La fila de UniWorks empieza vacía y se rellena celda a celda: Sí → Sí · x-* → Uniforme → ETag (≈ 2,3 s). Remata con ella.

SI PREGUNTAN

¿Por qué FastAPI y no Django REST o Flask?

Integración **nativa con Pydantic** para validar y generar esquemas, y es **asíncrono** (ASGI) con OpenAPI automático.

→ Cómo encajan las piezas.

Arquitectura

Dos sistemas y un contrato como única superficie de acoplamiento.

QUÉ DECIR

- Se lee de **izquierda a derecha**: el esquema nace en el backend y lo **consume** el frontend.
- **uniback**: modelo SQLAlchemy → Pydantic + pistas `x-*` → **JSON Schema enriquecido**.
- **Contrato HTTP**: todo, éxito o error, en `{ content, count, issues }`; tres endpoints: esquema por entidad, **bundle** y **descubrimiento**.
- **ngt-gui**: EntityClient / SchemaService → `schemaToFormly()` → `<ngt-dynamic-form>`.
- El frontend **nunca conoce** la forma concreta de una entidad.

EN PANTALLA

uniback y ngt-gui están desde el inicio; se dibuja la flecha, entra el contrato desde la izquierda y luego la segunda flecha (≈ 1,4 s).

SI PREGUNTAN

¿El contrato es bidireccional?

El esquema no: lo define el backend y el frontend solo lo consume. El frontend no necesita saber nada de la entidad de antemano.

→ Qué hay dentro del modelo.

Núcleo polimórfico

Un núcleo que trata igual lo que es distinto.

QUÉ DECIR

- Toda entidad hereda de **FunctionalObject** y se asocia a un **ObjectType** (código numérico).
- Anotaciones, **ACL** y colecciones apuntan a «cualquier objeto funcional», sin conocer su clase.
- La autorización igual: identidades y roles heredan de **Authorizable**.
- **Plant** llega desde un plugin y **lo hereda todo** sin que el núcleo sepa que existe.
- Amplía el modelo del ITC; lo **consolidé con versionado e histórico** (SQLAlchemy-Continuum).

EN PANTALLA

El diagrama crece como un árbol: ObjectType → FunctionalObject → Screen → Collection y Plant; las referencias punteadas llegan al final (≈ 2,7 s).

SI PREGUNTAN

¿Cómo se sabe de qué tipo es cada objeto?
Por **object_type_id**: es el discriminador del polimorfismo, y en el bundle viaja como `x-discriminator`.

→ ¿Y cómo entra un dominio nuevo sin tocar el núcleo?

Plugins y micronúcleo

El dominio entra desde fuera: plugins sobre un micronúcleo.

QUÉ DECIR

- Reutilizable ⇒ el núcleo **no conoce ningún dominio** → **plugins**.
- Un plugin hereda de **UnibackPlugin** y sobrescribe solo sus hooks: modelos, routers (fábrica CRUD), widgets.
- **PlantsPlugin**: modelos, CRUD y widget de mapa, **sin tocar una línea del núcleo**.
- **Micronúcleo**: autenticación, ficheros, anotaciones, especies… pasan a ser plugins de serie.
- **Multinodo**: misma imagen Docker; `UNIBACK_NODE_TYPE` decide qué sirve cada nodo y **Traefik** enruta por prefijo. Un nodo nuevo entra **en caliente**.

EN PANTALLA

Primero los nodos; después Traefik aparece fundiéndose y traza la línea hacia ellos; al final entra **seedbeds** con el borde punteado en movimiento (≈ 2 s).

SI PREGUNTAN

¿Por qué Traefik y no NGINX?
Se empezó con NGINX, pero Traefik **descubre servicios dinámicamente**: un nodo nuevo se enruta sin reconfigurar nada.

→ Cómo organicé el trabajo.

Metodología

Iterativo incremental: cuatro incrementos.

QUÉ DECIR

- **Iterativo incremental**, acordado con la empresa: cada incremento entrega una versión funcional y se replanifica.
- Encaja con el problema: **absorbe la incertidumbre** de un modelo que evoluciona. Descarté **Scrum**: mi línea de trabajo era **independiente**.
- Incremento 1: entorno reproducible con **Docker Compose** (Windows, macOS, Linux) y núcleo de modelos con versionado.
- Contorno = estimado; barra = real. **265 h estimadas → 290 reales**: desviación en los incrementos 1–3 (curva de aprendizaje del heredado); el 4 por debajo.

EN PANTALLA

Las barras reales crecen sobre las estimadas: verdes mientras caben, amarillas al alcanzarlas y rojas al pasarse; las horas reales y el total (290) suben a la vez (≈ 3,3 s).

SI PREGUNTAN

¿Por qué te desviaste?
Curva de aprendizaje del **código heredado** en los tres primeros incrementos. El cuarto se cerró con **34 h de 50**.

Incremento 2 · Contrato

Una sola forma para cualquier respuesta, también los errores.

QUÉ DECIR

- La base del ITC tenía envoltorio solo en lo generado por la fábrica: los **37 endpoints a mano** devolvían formas distintas, sin manejo global de errores.
- Ahora es **universal**: `content`, `count`, `issues`… **también los errores**: manejadores globales y **catálogo de 7 códigos**.
- Fábricas CRUD generalizadas: de ≈ 20 a **61 entidades**, **6 endpoints** cada una.
- **Descubrimiento**: el sistema es autodescriptivo. **EntityClient** en Angular, capa **aditiva**.

EN PANTALLA

Primero entra la respuesta de **éxito** (0,3 s) y después la de **error** (1,3 s). Señala que tienen la misma forma.

SI PREGUNTAN

¿Rompiste el código heredado?

No: la capa nueva es **estrictamente aditiva**; el servicio antiguo sigue devolviendo el cuerpo crudo y el interceptor de errores es **opt-in**.

→ La pieza principal: del esquema al formulario.

Incremento 3 · Formularios

Del esquema al formulario, sin escribir HTML. Principal aportación.

QUÉ DECIR

- **Backend**: SQLAlchemy → Pydantic inyecta por columna una **pista** según su tipo: booleano → checkbox, fecha → datepicker, FK → select con **carga diferida** que sabe a qué endpoint pedir.
- Son **valores por defecto**: lo declarado a mano **siempre gana**. Viajan dentro del esquema → **sobreviven al cacheo**.
- **Frontend**: `schemaToFormly()`, **conversor puro** → Formly con validadores y visibilidad condicional.
- Lo que escribe el desarrollador: **una etiqueta y la ruta**. El componente descubre, pide el esquema, monta, valida y envía.

EN PANTALLA

La flecha y `schemaToFormly()` aparecen con un barrido de izquierda a derecha (0,4 → 1,2 s).

SI PREGUNTAN

¿Limitaciones del conversor?

Procesa propiedades de primer nivel y sus hijos inmediatos; los overrides son por clave de primer nivel. Overrides por **path** y `oneOf` quedan como **trabajo futuro**.

→ Y para servirlos todos de una vez: el bundle.

Incremento 4 · Esquemas

Todos los esquemas, en un bundle versionado.

Esta diapositiva avanza paso a paso: 4 pulsaciones de → antes de pasar a la demo.

QUÉ DECIR

- **Sin resaltar**: cada fábrica registra su entidad en un **registro global** → **bundle**, un documento con todos los esquemas.
- → **1 · Versión = ETag**: hash del contenido; si nada cambia, **304** y el cliente usa su caché.
- → **2 · Polimorfismo**: la raíz de una jerarquía incluye sus subclases como `oneOf` con discriminador.
- → **3 · Relaciones**: los `relationship()` del ORM viajan como `x-relationships`.
- → **4 · Determinista**: mismo modelo, misma versión. Valor: un script externo pide el bundle y genera **tipos TS, validadores o documentación**.

EN PANTALLA

Cada → resalta un punto y sus líneas de código (el resto se atenúa). En el 4 el hash se borra y se reescribe idéntico. La 5.ª → pasa a la demo; ← retrocede un paso.

SI PREGUNTAN

¿Para qué sirve el ETag?

Para no volver a descargar el bundle si el modelo no ha cambiado: el servidor responde **304** y el cliente usa su caché.

→ Ahora, verlo funcionando.

Demo 1 · Uso general

Nada de lo que se ve está escrito a mano para cada entidad.

QUÉ DECIR

- Recorre la **autenticación**, la navegación por las **entidades descubiertas** y el **CRUD generado**.
- La lista, el alta y la edición **salen del esquema** de cada entidad.
- **Ninguna pantalla** está escrita a mano por entidad.

EN PANTALLA

Vídeo de **55 s**: arranca solo al entrar. **P** pausa o reanuda. Habla por encima, sin prisa.

SI PREGUNTAN

¿Y si una entidad necesita una pantalla especial?

Las pistas se sobrescriben por columna (lo manual **siempre gana**) y la capa genérica **convive** con el código escrito a mano.

→ Lo más llamativo: añadir un dominio en caliente.

Demo 2 · Un plugin en caliente

Un dominio nuevo sin reiniciar nada ni tocar el frontend.

QUÉ DECIR

- Con el sistema en marcha levanto con **Docker Compose** el nodo **seedbeds**, un plugin independiente.
- **Traefik** lo detecta, el **descubrimiento** lo incorpora y sus entidades **aparecen en la interfaz**.
- **Sin reiniciar** nada y **sin tocar el frontend**.

EN PANTALLA

Vídeo de **34 s**. Es corto: deja que se vea el momento en que aparece el menú del semillero.

SI PREGUNTAN

¿Cómo sabe la interfaz que el nodo existe?

El nodo emite un **latido en Redis**; mientras está vivo, la navegación muestra su sección. Si se para, desaparece al expirar el latido.

→ Dos herramientas construidas encima del framework.

Demo 3 · Pestañas y asistente de IA

Ventanas con pestañas y un asistente de IA que es un plugin más.

QUÉ DECIR

- **Ventanas y pestañas**: varias entidades abiertas a la vez.
- **Asistente de IA**: un plugin más; modelos **locales (LM Studio)** o **en la nube**.
- Consulta el **esquema** con una herramienta y propone altas bien formadas, de una o varias filas.
- **Proponer y confirmar**: nada se escribe sin aceptación, y siempre **con los permisos de la sesión**.

EN PANTALLA

Vídeo de **1 min 13 s**, el más largo. Si vas justo de tiempo, comenta mientras corre y pasa con →.

SI PREGUNTAN

¿Qué modelos admite el asistente?

Está preparado para **gemma 4**, **Claude**, **OpenAI** y compatibles; en local, servidos con LM Studio.

→ ¿Cuánto se ahorra? Los números.

Resultados

Una entidad nueva: ≈ 35 líneas de backend y ninguna de frontend.

QUÉ DECIR

- La cifra clave: una entidad nueva = **≈ 35 líneas de backend** y **0 de frontend**.
- **Medido en el código heredado**, antes: **≈ 135** de backend (modelo + router a mano) y **≈ 250** de frontend (formulario + métodos del servicio).
- Acumulado en las **61 entidades**: antes **8235** y **15 250**; con UniWorks **2135** y **0** → **−74 %** backend, **−100 %** frontend.
- Abajo: **366 endpoints** generados (6 × 61) frente a **178 métodos** a mano; **39** extensiones `x-*`; **41 tests** (45 %) + **17** del conversor.

EN PANTALLA

Las cuatro gráficas crecen a la vez, un peldaño por entidad, hasta 61 (≈ 4,8 s); después aparecen **−74 %** y **−100 %**. Espera a las insignias para dar el porcentaje.

SI PREGUNTAN

¿Cómo mediste el «antes»?

Backend: routers manuales de la base del ITC (`identity_store`, `hierarchies`): ≈ 24 líneas por operación × 4 + cabecera, más el mismo modelo. Frontend: mediana de **14 formularios** reales de la app heredada (229) + 4 métodos del `BackendService` (≈ 20). Sin blancos ni comentarios; **conservador**: sin esquemas Pydantic.

→ Pero lo más importante no es una cifra.

Antes y después

Exponer y editar una entidad, antes y después.

QUÉ DECIR

- Antes: router a mano, validación a mano, errores **ad hoc**, un método por operación y formulario **campo a campo**.
- Ahora: **una fábrica**, validación **derivada del ORM**, **envoltorio uniforme**, **cliente genérico** y **una etiqueta**.
- Última fila = el problema de partida: un cambio en el modelo **ya no se propaga a mano a tres capas: se propaga solo**.
- El esfuerzo **deja de crecer** con el número de entidades y con la frecuencia de cambios.

EN PANTALLA

La columna «Con UniWorks» se rellena fila a fila; la última llega con algo más de pausa (≈ 2,9 s). Léela al final.

SI PREGUNTAN

¿Y si un cambio no lo cubre el framework?

Lo manual siempre gana: se sobrescriben las pistas o se escribe código propio, que **convive** con lo generado.

→ Cierro con las conclusiones.

Conclusiones y trabajo futuro

El coste de un modelo que cambia deja de crecer.

QUÉ DECIR

- Partía del coste de **mantener sincronizados** backend, comunicación e interfaz; se reduce **drásticamente** si el **ORM es la única fuente de verdad**.
- ✓ **5 objetivos** con pruebas · ✓ contrato y cadena que **desacoplan del dominio** · ✓ validado en necesidades **reales del ITC**.
- **ODS**: alto con el **9**; medio con el **15** y el **8**.
- **Futuro**: codegen desde el bundle, migrar el servicio heredado al cliente genérico, `oneOf` y overrides por ruta en el conversor, criptografía.
- **IA**: apoyo para explorar el heredado y depurar, **bajo mi supervisión**; el diseño es mío.

EN PANTALLA

Se subraya «deja de crecer», entran las tres conclusiones con ✓ y después el trabajo futuro desde la derecha (≈ 3,5 s).

SI PREGUNTAN

¿Qué papel tuvo la IA?

Apoyo en tareas repetitivas, explorar el código heredado, sugerir refactorizaciones y sobre todo **depurar**; todo **validado por mí**. No tomó decisiones de diseño, integración ni verificación.

→ Agradecimientos y preguntas.

Cierre

Gracias. Quedo a disposición del tribunal.

QUÉ DECIR

- **Muchas gracias** por su atención.
- Gracias a **Rafael Nebot** y al **ITC** por la oportunidad, y a **Marilola Afonso** por su apoyo.
- El código está en el **repositorio** que aparece en pantalla.
- Quedo a disposición del tribunal para sus **preguntas**.

EN PANTALLA

La planta vuelve a crecer. Respira, mira al tribunal y espera a la primera pregunta.

SI PREGUNTAN

Si no sabes la respuesta

«No lo he evaluado en este trabajo; lo plantearía como trabajo futuro». Mejor que improvisar. Ten a mano la 17 (cifras) y la 7 (arquitectura).

→ Turno de preguntas.